

# Eco Font Converter

Save up to 33% printer ink by converting documents to stroke-only text rendering — the same technique used by the Ryman Eco font.

Instead of filling each character solid, it draws only the outlines. The result looks identical on screen but uses significantly less ink when printed.

---

## Requirements

- Python 3.10+
- pip

Install core dependencies:

```
pip install pypdf reportlab
```

Optional (only needed for .docx files):

```
pip install python-docx
```

Or let the script install them automatically with `-y`.

---

## Usage

### Single file

```
python eco_font_converter.py report.pdf python eco_font_converter.py notes.docx  
python eco_font_converter.py README.md
```

Output is saved next to the input as `<name>_eco.pdf`.

### Custom output path

```
python eco_font_converter.py report.pdf --output report_print.pdf python  
eco_font_converter.py report.pdf -o report_print.pdf
```

### Batch — multiple files or globs

```
python eco_font_converter.py *.pdf python eco_font_converter.py *.pdf *.docx *.md  
python eco_font_converter.py file1.pdf file2.docx
```

## Batch — entire folder

```
python eco_font_converter.py --folder ./reports python eco_font_converter.py -f
./reports --output-dir ./eco_output
```

Processes all .pdf, .docx, .md, and .markdown files in the folder.

## Auto-install dependencies

```
python eco_font_converter.py report.pdf --auto-install python eco_font_converter.py
report.pdf -y
```

Installs any missing packages via pip before converting.

## Preview without converting

```
python eco_font_converter.py *.pdf --dry-run
```

Shows what files would be converted and where output would go, without touching anything.

## Interactive mode

Run with no arguments and the script will prompt you to enter a file path:

```
python eco_font_converter.py
```

Useful when launching from a desktop shortcut or double-clicking the file.

---

## All options

| Flag           | Short | Default              | Description                                 |
|----------------|-------|----------------------|---------------------------------------------|
| --output       | -o    | <name>_eco.pdf       | Output path (single file only)              |
| --output-dir   | -d    | same folder as input | Output folder for batch mode                |
| --folder       | -f    | —                    | Process all supported files in a folder     |
| --stroke-width | -s    | 0.7                  | Thickness of character outlines (see below) |
| --auto-install | -y    | off                  | Auto-install missing pip packages           |
| --dry-run      | —     | off                  | Preview without converting                  |

---

## Stroke width guide

The stroke width controls how thick the character outlines are.

| Range | Effect |

|——|——|

| 0.4 - 0.6 | Very light — maximum ink saving, may look faint |

| 0.7 | Default — good balance of readability and savings |

| 0.8 - 1.0 | More visible — easier to read, less saving |

```
# Lighter (save more ink) python eco_font_converter.py report.pdf --stroke-width 0.5
# Heavier (easier to read) python eco_font_converter.py report.pdf --stroke-width 0.9
```

---

## Supported formats

| Format | Notes |

|——|——|

| .pdf | Injects stroke-only rendering directly into the PDF stream |

| .docx | Extracts text and headings, renders to eco PDF |

| .md / .markdown | Parses Markdown including headings, bold, italic, code blocks, and lists |

---

## How it works

PDF files contain content streams — sequences of drawing commands. Text is typically rendered with mode 0 (fill). This script:

Decompresses each content stream

Removes any existing text rendering mode commands

Wraps every BT...ET 0 Tr text block with (stroke-only mode) and sets a stroke width

Recompresses and writes the result

For DOCX and Markdown, the script first renders the content to a temporary PDF using ReportLab, then applies the same eco transformation.

---

## Architecture

```
flowchart TD
    CLI["CLI - main()"] --> RESOLVE["_resolve_inputs()\nExpand globs · scan folder · dedup"]
    RESOLVE --> SUPPORTED["supported"]
    RESOLVE --> UNSUPPORTED["unsupported"]
    UNSUPPORTED --> SKIP["Skipped - user notified\n.env, .py, .jpg ..."]
    CLI --> DEPS["ensure_deps()\nCheck / auto-install pip packages"]
    DEPS --> FONTS["_register_fonts()\nResolve system TTF → EcoSans / EcoMono\nFallback: Helvetica"]
```

```

/ Courier"] FILES --> ROUTER{"Format?"} ROUTER -->|.pdf| PDF["convert_pdf()"] ROUTER
-->|.docx| DOCX["convert_docx()"] ROUTER -->|.md / .markdown|
MD["convert_markdown()"] DOCX --> RL_DOCX["ReportLab\nRender paragraphs +
tables\nPreserve bold / italic"] MD --> RL_MD["ReportLab\nParse headings,
lists,\ncode blocks, inline markup"] RL_DOCX --> TMP["Temporary PDF"] RL_MD --> TMP
TMP --> PDF PDF --> STREAM["Per-page content streams"] STREAM -->
INJECT["_inject_eco_into_stream()]\nø Strip existing Tr commands\nø Wrap BT...ET ø Tr with 1
Tr\nø Prepend stroke width"] INJECT --> COMPRESS["zlib recompress → FlateDecode"]
COMPRESS --> OUT["Output PDF\n<name>_eco.pdf"] OUT --> SUMMARY["Summary\nFiles
converted • sizes • errors"]

```

---

## Examples

```

# Convert one PDF, default settings python eco_font_converter.py quarterly_report.pdf
# Convert all PDFs in a folder, save to a separate output folder python
eco_font_converter.py --folder ./invoices --output-dir ./invoices_eco # Very light
strokes for maximum ink saving python eco_font_converter.py brochure.pdf -s 0.5 #
Convert a Word doc, auto-installing dependencies python eco_font_converter.py
proposal.docx -y # Preview what a glob would convert python eco_font_converter.py
*.docx --dry-run

```